

MANUEL D'UTILISATION

Expertise Digitale

Mode d'emploi complet du système de prospection
automatisée : architecture, fonctionnement, et
commandes de vérification

Document généré à partir de l'analyse réelle du système · Usage strictement personnel

Sommaire

1. Résumé — à quoi sert ce système	3
2. Architecture du système	4
3. Comment fonctionne le pipeline commercial	6
4. Structure des fichiers du projet	8
5. Commandes de vérification et de suivi	9
5.1 Est-ce que tout tourne ?	9
5.2 Suivre les journaux (logs) en direct	9
5.3 Interroger l'API	9
5.4 Voir la base de données (Supabase)	10
5.5 Interfaces web (Streamlit / n8n)	10
5.6 Journaux sur le disque	10
6. Déclencher une action manuellement, sans risque	11
7. Avertissements de sécurité	12
8. État d'avancement — ce qui reste à finaliser	13
9. Glossaire	14
10. Foire aux questions / dépannage	15

1. Résumé — à quoi sert ce système

Expertise Digitale est un système entièrement automatisé qui trouve, contacte et suit des artisans du bâtiment n'ayant pas de site web, dans la Métropole de Lyon. Il fonctionne seul, en continu, 24h/24, sans intervention manuelle nécessaire au quotidien.

En résumé, le système effectue ces actions à ma place :

1. Il repère des entreprises du BTP sans présence digitale via des données publiques officielles (le registre SIRENE).
2. Il vérifie que chaque contact (email, téléphone, site) est réel avant tout envoi.
3. Il rédige un e-mail de prospection personnalisé grâce à une intelligence artificielle installée localement (aucun coût par message).
4. Il envoie cet e-mail, puis relance automatiquement en l'absence de réponse.
5. Il surveille ma boîte mail pour détecter les réponses positives, et enchaîne automatiquement : présentation détaillée, formulaire de qualification, devis, signature électronique, paiement.

Trois offres commerciales sont actives dans ce système :

- (a) génération de leads qualifiés (paiement au contact ou abonnement — tarif à finaliser) ;
- (b) site vitrine « clé en main » à 990 € TTC avec devis, signature et paiement intégrés ;
- (c) revente de fichiers de prospection B2B (listes d'entreprises qualifiées).

Point d'attention légal : aucune structure légale (SIRET, forme juridique) n'est encore enregistrée à ce jour. Des prospects réels la demandent déjà — c'est une priorité à traiter rapidement (voir section 8).

2. Architecture du système

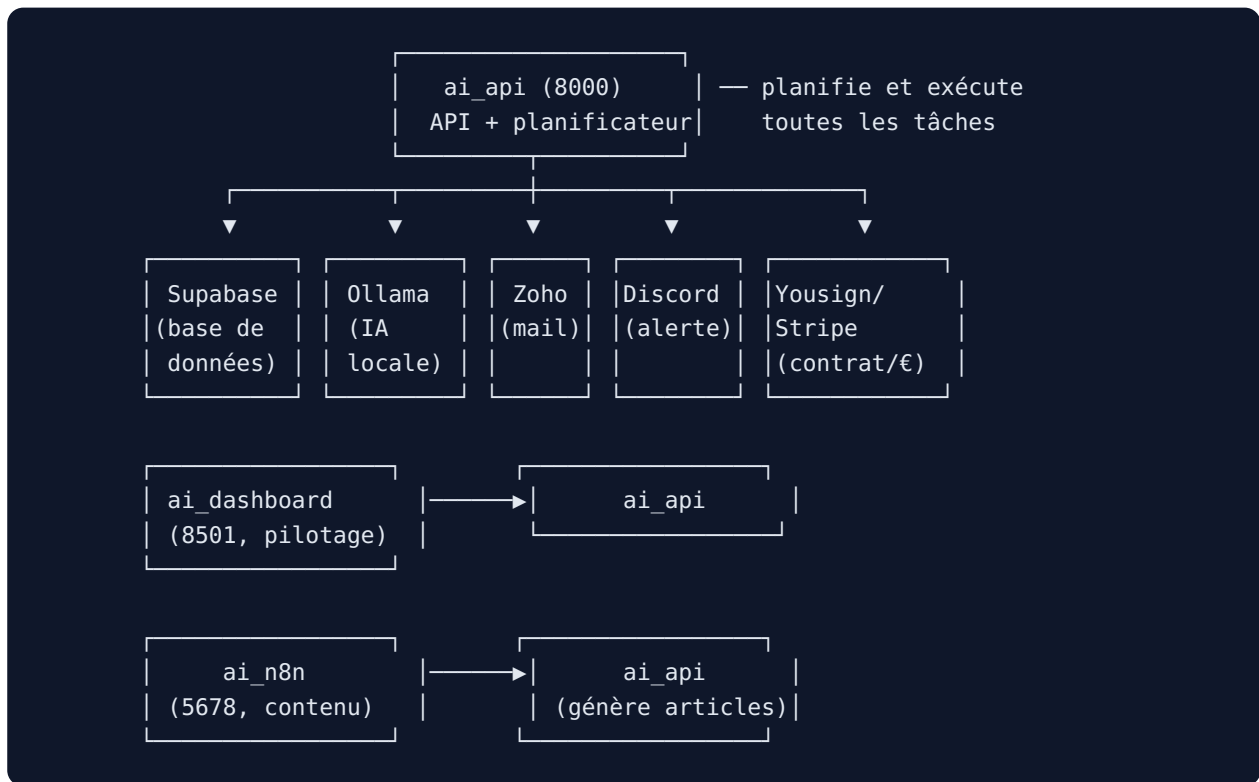
Le système repose sur **4 programmes indépendants** ("conteneurs"), qui tournent ensemble en permanence sur mon ordinateur/serveur, et redémarrent automatiquement en cas de panne.

Composant	Rôle	Adresse (sur mon ordinateur)
ai_api	Le « cerveau » : gère toutes les données et exécute automatiquement les tâches planifiées (scraping, envoi d'e-mails, relève des réponses, relances)	http://localhost:8000
ai_ollama	Intelligence artificielle installée localement, qui rédige les e-mails de prospection et les rapports	http://localhost:11435
ai_dashboard	Interface web de pilotage (tableau de bord visuel)	http://localhost:8501
ai_n8n	Outil d'automatisation additionnel, utilisé pour la génération planifiée d'articles de blog	http://localhost:5678

Le système communique aussi avec des services extérieurs :

Service externe	Utilisé pour
Supabase	Base de données où sont stockés tous les prospects, contrats et réponses
Zoho Mail	Envoi (SMTP) et réception (IMAP) des e-mails de prospection
Discord	Alertes automatiques : lead intéressé détecté, panne du système
Yousign	Signature électronique des contrats (<i>actuellement en mode test, voir section 8</i>)
Stripe	Encaissement des paiements clients

Schéma général

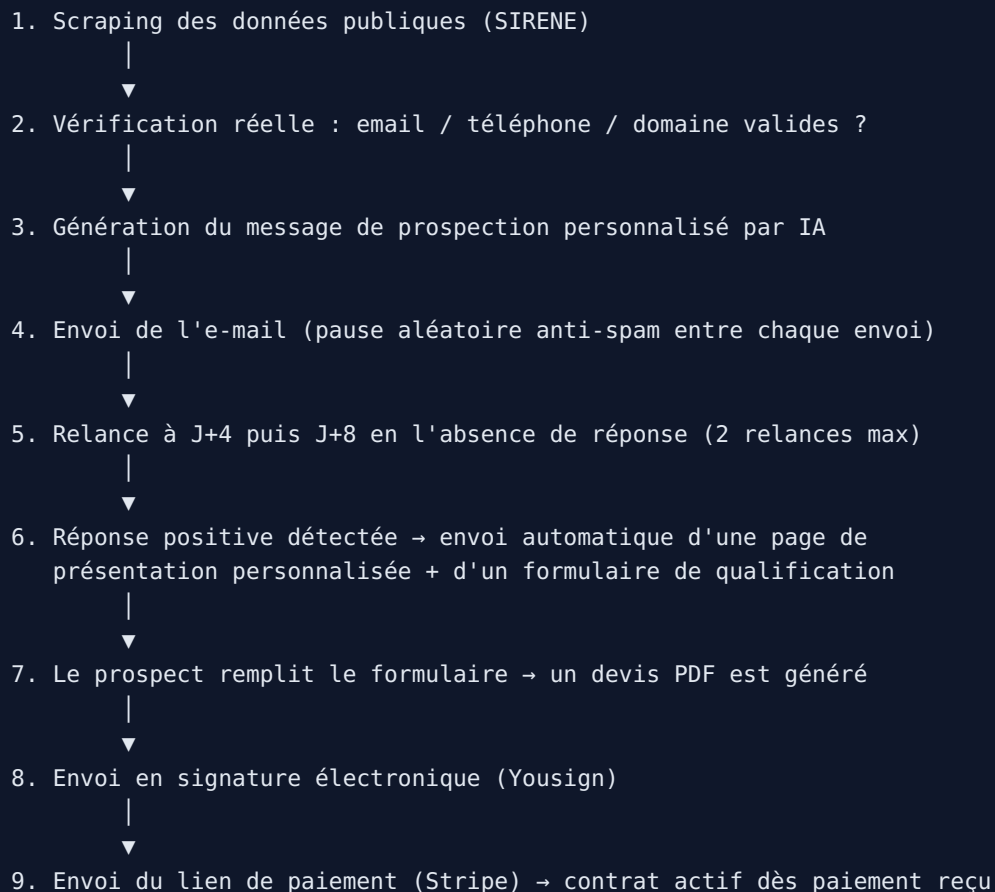


Ce qui tourne automatiquement, sans que j'aie besoin d'y penser

Tâche automatique	Fréquence	Ce qu'elle fait concrètement
Vérification des réponses e-mail	toutes les 5 minutes	Se connecte à la boîte mail, détecte les réponses positives/négatives, met à jour le prospect, alerte si besoin
Traitement IA des nouveaux prospects	toutes les heures	Génère le pitch commercial de chaque nouveau prospect scrapé
Campagne de prospection + rapport	tous les jours à 20h	Envoie les e-mails de prospection du jour, puis m'envoie un rapport de synthèse
Relance des prospects sans réponse	tous les jours à 9h30	1ère relance à J+4, 2e et dernière à J+8, puis abandon définitif
Surveillance de la santé du système	toutes les 15 minutes	Vérifie que la base de données et l'IA répondent ; alerte uniquement en cas de changement d'état

3. Comment fonctionne le pipeline commercial

Voici, étape par étape, tout le trajet parcouru automatiquement par un prospect, de sa découverte jusqu'au contrat signé et payé :



Le rôle de chaque script dans ce trajet

Fichier	Ce qu'il fait
<code>scraper_batiment.py</code>	Va chercher les entreprises sur les données publiques officielles (SIRENE)
<code>email_enricher.py</code>	Vérifie et déduit les emails/téléphones réels (jamais d'invention de contact)
<code>lead_worker.py</code>	Génère le pitch commercial par IA et envoie le premier e-mail
<code>ceo_agent.py</code>	Lance la campagne d'envoi du jour et le rapport hebdomadaire
<code>mail_processor.py</code>	Surveille la boîte mail et détecte les réponses des prospects
<code>relance_prospects.py</code>	Gère les relances automatiques programmées
<code>export_leads_commerciaux.py</code>	Génère les fichiers de prospection destinés à la vente

4. Structure des fichiers du projet

Le projet se trouve sur mon ordinateur à l'emplacement `~/ai-company` (autrement dit `/home/mohamed/ai-company`).

```
ai-company/
├─ docker-compose.yml      → définit les 4 conteneurs et comment ils communiquent
├─ .env                    → tous les mots de passe/clés (NE JAMAIS PARTAGER)
├─ api/
│   └─ main.py              → le cœur : routes web + planificateur automatique
├─ dashboard/
│   └─ app.py               → interface web de pilotage (Streamlit)
├─ scraper_batiment.py     → recherche des entreprises (données SIRENE)
├─ email_enricher.py       → vérifie/déduit les emails et téléphones réels
├─ lead_worker.py          → génère le pitch IA et envoie le 1er e-mail
├─ ceo_agent.py            → campagne du jour + rapport hebdomadaire
├─ mail_processor.py       → surveille la boîte mail, détecte les réponses
├─ relance_prospects.py    → gère les relances automatiques
├─ export_leads_commerciaux.py → génère les fichiers de prospection à revendre
├─ sql/init.sql            → structure complète de la base de données
├─ agents/workers/         → génération d'articles de blog par IA
├─ cron.log / crm.log / erreurs_envoi.log → journaux d'activité sur le disque
└─ leads.json              → fichier local temporaire de scraping
```

Explorer cette structure moi-même, à tout moment

Dans un terminal, à la racine du projet :

```
cd ~/ai-company
ls -la          # liste tous les fichiers et dossiers du premier niveau
tree -L 2       # vue arborescente sur 2 niveaux (si l'outil "tree" est
installé)
find . -maxdepth 2 -not -path '*/venv*' -not -path '*/__pycache__*' # équivalent
sinon
```


5. Commandes de vérification et de suivi

Toutes les commandes ci-dessous s'exécutent dans un terminal, après s'être placé dans le dossier du projet : `cd ~/ai-company`.

5.1 Est-ce que tout tourne ?

```
docker compose ps
```

Je dois voir `Up ... (healthy)` pour les 4 services.

5.2 Suivre les journaux (logs) en direct

```
# Tout ce que fait le cerveau (API + planificateur), en direct
docker logs -f ai_api

# Uniquement les actions automatiques du planificateur
docker logs ai_api --since 1h | grep "Cron :"
```

```
# Uniquement la veille des réponses e-mails
docker logs ai_api --since 6h | grep -i "mail_processor\|réponse positive\|
désinscription\|nouveau message"
```

```
# Logs du tableau de bord
docker logs -f ai_dashboard
```

5.3 Interroger l'API directement

```
# Santé des dépendances (Supabase, Ollama, config Zoho/Discord)
curl -s http://localhost:8000/health | python3 -m json.tool
```

```
# Statistiques globales (nécessite la clé API du .env)
API_KEY=$(python3 -c "from dotenv import dotenv_values; print(dotenv_values('.env')
['API_SECRET_KEY'])")
curl -s -H "X-API-Key: $API_KEY" http://localhost:8000/stats | python3 -m json.tool
```

```
# Liste des prospects en base
curl -s -H "X-API-Key: $API_KEY" http://localhost:8000/leads | python3 -m json.tool |
less
```

5.4 Voir la base de données directement (Supabase)

```
set -a && source .env && set +a
curl -s "$SUPABASE_URL/rest/v1/leads?
select=company,status,contacted,contacted_at&order=created_at.desc&limit=15" \
-H "apikey: $SUPABASE_KEY" -H "Authorization: Bearer $SUPABASE_KEY" | python3 -m
json.tool
```

Remplacer `leads` par `contracts`, `intake_responses` ou `ceo_reports` pour consulter les autres tables. Il est aussi possible d'ouvrir directement l'interface web Supabase (Table Editor) pour parcourir visuellement les données.

5.5 Interfaces web (Streamlit / n8n)

Tableau de bord (Streamlit) : ouvrir `http://localhost:8501` — vue d'ensemble, liste des prospects, contenus générés, et un bouton pour déclencher une action manuellement en toute sécurité.

n8n (automatisation de contenu) : ouvrir `http://localhost:5678` — identifiants dans les variables `N8N_USER` / `N8N_PASSWORD` du fichier `.env`.

5.6 Journaux sur le disque (en dehors de Docker)

```
tail -f ~/ai-company/cron.log           # historique long terme du planificateur
tail -f ~/ai-company/crm.log           # traitement des réponses des prospects
tail -f ~/ai-company/erreurs_envoi.log # échecs d'envoi d'e-mails
```

6. Déclencher une action manuellement, sans risque

La méthode la plus sûre consiste à utiliser le tableau de bord web :

1. Ouvrir `http://localhost:8501` dans un navigateur.
2. Aller à la section « Actions de l'agent ».
3. Choisir l'action désirée : scraping, traitement des leads, vérification des mails, campagne + rapport, ou relances.
4. Cliquer sur « Déclencher ».

C'est la seule méthode recommandée pour tester une action à la main : elle passe par le même circuit que l'automatisation normale, sans risque de double-exécution ni d'erreur de configuration.

7. Avertissements de sécurité

Ne jamais lancer manuellement les commandes `python3 lead_worker.py` ou `python3 ceo_agent.py` « juste pour tester » : ces scripts envoient immédiatement de **vrais e-mails à de vrais prospects**, sans confirmation ni simulation.

Le fichier `.env` contient tous les mots de passe et clés secrètes du système (base de données, e-mail, paiement, signature électronique). Il ne doit **jamais** être partagé, copié dans un message, ou publié en ligne.

La clé Supabase utilisée est une clé « **service_role** » : elle donne un accès total (lecture ET écriture) à toute la base de données. À traiter comme un mot de passe root.

La clé `API_SECRET_KEY` protège toutes les actions sensibles de l'API : même niveau de confidentialité qu'un mot de passe.

Éviter de partager, en dehors d'un usage strictement personnel, le résultat des commandes qui affichent des données de prospects (elles contiennent potentiellement des données personnelles : noms, e-mails, téléphones).

8. État d'avancement — ce qui reste à finaliser

Signature électronique (Yousign) : fonctionne actuellement en mode test (« sandbox »). Aucune signature effectuée dans cet état n'a de valeur juridique — à basculer en production avant le premier vrai contrat.

Statut légal : aucune structure légale (SIRET, forme juridique, adresse de siège) n'est encore enregistrée. Des prospects réels la demandent déjà dans leurs échanges — c'est la priorité la plus urgente.

Configuration vestigiale : certaines variables de configuration (base Postgres classique, Redis) existent dans les fichiers mais ne sont reliées à aucun service actif — à ignorer ou nettoyer.

Donnée de test : un enregistrement de test (`__TEST_E2E_TUNNEL__`) est présent dans les données réelles et doit être ignoré dans tout comptage, voire supprimé.

9. Glossaire

Terme	Explication simple
Docker / conteneur	Une façon d'emballer un programme avec tout ce dont il a besoin pour tourner, de façon isolée et reproductible
API	Le « guichet » par lequel les différents programmes du système se parlent entre eux via internet
SIRENE	Le registre officiel et public de toutes les entreprises françaises
SMTP	Le protocole utilisé pour <i>envoyer</i> des e-mails
IMAP	Le protocole utilisé pour <i>lire/récupérer</i> les e-mails d'une boîte mail
Webhook	Une notification automatique envoyée par un service externe (ex : Stripe) dès qu'un évènement se produit (ex : paiement reçu)
Sandbox	Un environnement de test, séparé de la réalité, sans conséquence ni valeur légale
Base de données (Supabase)	L'endroit où sont stockées durablement toutes les informations (prospects, contrats...)
Ollama	Le logiciel qui fait tourner l'intelligence artificielle directement sur mon ordinateur, sans dépendre d'un service payant externe
Planificateur (scheduler)	Le mécanisme qui déclenche automatiquement les tâches à intervalles fixes, sans intervention humaine

10. Foire aux questions / dépannage

Le tableau de bord (localhost:8501) ne s'ouvre pas, que faire ?

Vérifier d'abord que les conteneurs tournent avec `docker compose ps`. Si `ai_dashboard` n'est pas « healthy », consulter ses logs avec `docker logs ai_dashboard`.

Comment savoir si un e-mail a bien été envoyé à un prospect ?

Consulter `erreurs_envoi.log` pour les échecs, ou interroger directement la fiche du prospect dans la base (section 5.4) : le champ `contacted_at` indique la date du dernier envoi.

Comment savoir si le système a bien détecté une réponse positive ?

Regarder les logs avec le filtre du point 5.2 (la mention « réponse positive détectée » apparaît dans le journal), ou consulter le statut du prospect en base (`status` passera à `interested` puis `presentation_envoyee`).

Le système semble en panne (alerte Discord reçue), que faire ?

Consulter `http://localhost:8000/health` (section 5.3) pour identifier quel service est en cause (Supabase ou Ollama), puis vérifier les logs du conteneur concerné.

Puis-je arrêter puis relancer le système sans tout casser ?

Oui : `docker compose down` puis `docker compose up -d` depuis la racine du projet. Les données restent en sécurité dans Supabase (base externe), rien n'est perdu localement.